CS 3300 AI Agent Workshop

Single Agent to Multi Agent Feature Development Demo Notes

We will add task priority to a working web application, first with one AI implementation agent and then with separate Planner, Developer, and Reviewer roles. Follow how the requirement becomes code, executed tests, review feedback, and a pull request.

This is a TA-led demonstration. Follow the shared screen, inspect the examples, and discuss the checkpoints. You do not need to write code, configure credentials, or make paid model calls during the session.

What you should be able to explain

- Identify each role's inputs and outputs, and explain which actions ordinary Python tools perform.
- Distinguish a model's claim of success from executed tests and evidence in a code diff.
- Explain how review feedback drives another attempt and why retries have a limit.
- Compare the added review and coordination with the additional model calls and time.

Session map

Minutes	What we will follow
0 to 10	Starting app, priority requirement, and execution structure
10 to 30	Single-agent implementation and inspection of its result
30 to 35	Discussion of coverage and review limitations
35 to 55	Planner, Developer, and Reviewer handoffs
55 to 65	Review feedback, new tests, and another attempt
65 to 70	GitHub issue approval and pull request workflow
70 to 75	Comparison and closing discussion

The presenter may adjust timing or show a saved example. Note whether each result is a live model run, a saved run, or a local mock of publication.

Workshop repository | Priority feature issue

1. Understand the starter codebase

The repository contains two related programs: the Task Tracker that users interact with, and the Python workflow that asks AI to modify it. The starter application is already implemented; our focus is the development workflow around one new feature.

Repository map

```
ta-workshop-starter/
  app.py                        # Flask routes and in-memory tasks
  templates/index.html          # Page and create-task form
  static/app.js                 # API requests and task rendering
  static/style.css              # Page styles
  tests/test_app.py             # Five starter backend tests
  main.py                       # Local single or multi menu
  orchestrator.py               # Role order and repair loop
  agents/
    single_agent.py             # One implementation call
    planner.py                  # Plan and file selection
    developer.py                # Proposed file replacements
    reviewer.py                 # Review status and feedback
  tools/
    llm.py                      # Model API adapter
    mcp_client.py               # Tool dispatcher
    mcp_server.py               # Registered tool functions
  docs/coding_guidelines.md     # Reusable team instructions
  .github/workflows/
    agent-feature.yml           # approved label trigger
  requirements.txt             # Python dependencies
```

What the model does and what Python does

An AI role is a model call with a particular job and supplied instructions. The model proposes a plan, file edits, or review feedback. The orchestrator is ordinary Python code that decides the stage order, calls tools, and checks whether the process may continue.

The tool module registers functions with Model Context Protocol (MCP), a standard interface for tools and context.

The coding guidelines file supplies reusable project context, such as naming rules and environment variables for credentials. The implementation and review roles receive it on each run. It is persistent guidance (long-term memory); the repository does not automatically learn a history of previous work.

Follow the demonstration

1. On the shared screen, locate app.py and the three folders agents, tools, and tests.
2. Open main.py and find the menu options. Then locate run_multi_agent_pipeline in the root-level orchestrator.py.

Browse this code revision

2. Establish the feature and the baseline

The Task Tracker uses Flask, HTML, CSS, JavaScript, and pytest. It can list, add, and delete tasks. Tasks live in memory and reset when the server restarts. The new priority field must travel through the entire request and response path.

```
Form selection -> JavaScript JSON -> POST /api/tasks -> backend storage -> JSON
response -> displayed task
```

The priority requirement

- Support Low, Medium, and High priority, with a choice when creating a task.
- Store and return priority in the backend, and display it in the browser.
- Use Medium when priority is omitted, and preserve existing behavior.

These grouped requirements correspond to the six items in the issue. A dropdown alone is insufficient: the submitted value must survive storage and appear in the response and task list.

Where the backend change belongs

Source app.py

Excerpt from the existing add_task route

```
new_task = {
    "id": next_id,
    "title": data['title']
}
tasks.append(new_task)
next_id += 1
return jsonify(new_task), 201
```

The current task object contains id and title. Priority needs to become part of task creation and returned data. The form in templates/index.html and the request/rendering code in static/app.js must also change.

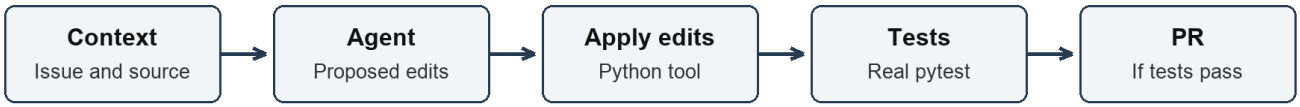
Follow the demonstration

1. Watch the TA run python app.py and open http://127.0.0.1:5000 on the shared screen. Watch a task being added and deleted; confirm that no priority selector exists yet.
2. Open Issue 1 and identify the omitted-priority default.
3. Watch python -m pytest tests/ -v. Read the five starter test names: page, listing, creation, missing title, and deletion. There are no priority tests yet.

Pause and discuss If the selector shows High but the backend always stores Medium, which requirements are still unmet?

3. Run the single agent workflow

The single-agent flow gives one implementation role the requirement, repository source, and coding guidelines. It returns edits and a summary. The tools apply the edits and run actual tests; passing tests allow the publication step.



One model call proposes code; ordinary tools perform the actions.

Read the implementation handoff

Source agents/single_agent.py

Selected lines from run_single_agent with intervening code omitted

```
result = call_llm_json(prompt)
if "edits" in result:
    execute_mcp_tool("apply_edits", {"edits": result["edits"]})
...
tests = execute_mcp_tool("run_test_suite")
```

Each item in edits contains file_path and content, where content is the complete replacement file. The model's summary describes its intention; the diff shows the changes that were actually applied. The test tool runs pytest independently of any claim made by the model.

Why the first result needs inspection

The five existing tests check old behavior. They can pass even when coverage for priority is missing. This single-agent path has no separate Reviewer and no automatic repair cycle. Its prompt does not explicitly require new tests.

Follow the demonstration

1. Open the prompt in agents/single_agent.py. Find Issue Requirement, Current File Contents, Coding Guidelines, edits, and summary.
2. Watch the TA run python main.py and enter 1. Follow the implementation message, Test Status, pytest output, and publication message.
3. Inspect git diff --stat and git diff. Find the backend, form, and JavaScript changes, and check whether tests/test_app.py also changed.

Locally, the issue requirement is built in and PR publication is mocked. Model calls, file edits, and tests are real. PR creation mocked locally therefore does not mean that a GitHub PR exists.

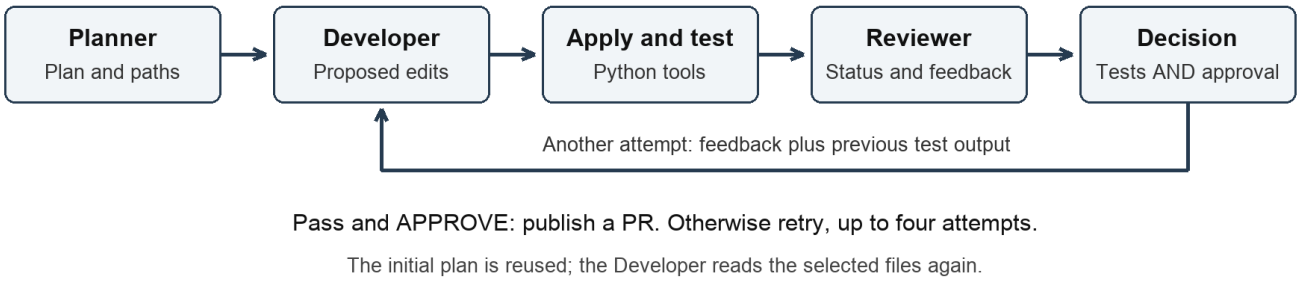
Pause and discuss If the old tests pass but the new feature has no new tests, what evidence is still missing?

4. Separate planning development and review

We now give planning and review their own model calls. This makes the handoffs visible and lets review findings guide a later implementation attempt. Python still determines the order. The same model adapter is used for the roles.

Role	Inputs	Output

Planner	Issue and repository structure	files_to_modify and plan
Developer	Issue, plan, selected files, guidelines, and any feedback	edits and summary
Reviewer	Issue, diff, actual test results, and guidelines	status and feedback



See how the plan becomes Developer context

Source orchestrator.py

Selected lines with omissions and line wrapping adjusted for readability

```
plan = create_plan(issue, repo)
files_needed = plan.get("files_to_modify", [])
...
files_content = execute_mcp_tool(
    "get_files", {"file_paths": files_needed}
)
implementation = implement_feature(
    issue, plan, files_content, memory_context,
    dev_feedback, test_output
)
```

Follow the demonstration

1. Watch the TA switch to a separate, clean checkout. Confirm that priority is still absent so both flows start from the same app.
2. In agents/planner.py, locate the instruction to exclude tests from the initial plan. In agents/reviewer.py, locate the instruction to request tests for new feature logic.
3. Watch the TA run python main.py and choose 2. Follow Planner, Developer, tests, and Reviewer in the terminal.

5. Use review feedback to improve the result

The demo deliberately asks the Planner to leave tests out of its first plan. The Reviewer should then request coverage for the new feature. A later Developer call receives the feedback and previous test output, can add tests, and is checked again.

Trace the actual feedback and approval condition

Source orchestrator.py

Selected lines with logging and publication details omitted

```
tests = execute_mcp_tool("run_test_suite")
test_output = tests["output"]
diff = execute_mcp_tool("get_repository_diff")
review = review_feature(issue, plan, diff, tests, memory_context)
status = review.get("status")
dev_feedback = review.get("feedback")
...
if tests["passed"] and status == "APPROVE":
    ...
```

Reviewer statuses are APPROVE, REQUEST_CHANGES, and REQUEST_MORE_TESTS. The condition uses AND: failed tests block publication even if the Reviewer approves, and a rejecting Reviewer blocks publication even if tests pass.

Recognize the intended first attempt

```
--- Attempt 1 of 4 ---
Test Status: PASSED
Review Status: REQUEST_MORE_TESTS
Feedback: <reviewer explains which tests are missing>
```

The intended teaching example takes two attempts, but model output can vary. The code allows up to four Developer attempts, reuses the initial plan, and stops when the checks succeed or the attempt limit is reached.

Follow the demonstration

1. Read the actual first review. Identify the missing behavior or test it requests.
2. In agents/developer.py, find if feedback and the instruction to fix the implementation or add tests to tests/test_app.py. Watch the next Developer, test, and Reviewer calls.
3. After the run, inspect the diff and pytest output. Locate an assertion for a chosen priority and one for an omitted priority defaulting to Medium.
4. Watch the resulting browser app create High and Low tasks and delete a task. A form that always submits a value does not exercise the backend default; inspect the omitted-value request or test too.

Pause and discuss Name one concrete change caused by feedback. Why must another attempt remain blocked if tests pass but review still requests changes?

6. Follow GitHub automation and compare results

GitHub connects the requirement to a reviewable contribution. An approved issue starts the Action, the multi-agent workflow edits and tests code, and a successful result becomes a pull request. Human review and the merge decision remain separate.

Find the issue label trigger

Source .github/workflows/agent-feature.yml

Selected workflow lines; the condition belongs to the run-agent job

```
on:
  issues:
    types: [labeled]
...
if: github.event.label.name == 'approved'
```

Behavior	Local main.py	GitHub Action
Requirement	Built-in priority example	Fetches the issue
Model and tests	Real model calls and pytest	Real model calls and pytest
Publication	Mock PR message	Branch and regular PR targeting main
Flow selection	Choose single or multi	Runs multi only

Follow the demonstration

1. Watch the TA demonstrate the approved label in a prepared repository, or inspect a clearly labeled saved run. Students do not need to trigger paid runs.
2. Open the shown Actions run. Find checkout, dependency installation, and the orchestrator invocation with the issue number.
3. Open the resulting PR and Files changed. Compare the feature and test changes with the issue, then inspect the test evidence and browser behavior.

The successful publisher creates feature-issue-N and opens a regular PR targeting main. It does not merge automatically. A local mock message does not verify GitHub runner availability or publication permissions.

Compare the evidence from both flows

- Did each result meet the same issue from the same initial app? Point to code, tests, and browser behavior.
- What did review catch, and what changed in response? What would you still check before merging?
- What extra work did the roles introduce? Single normally uses one model call; multi uses three for one attempt and five for two, plus tool execution and waiting time.

7. Instructions for Assignment Setup

After the demo, use the following steps to set up and run the workflow on your own GitHub repository:

1. **Download the starter repository**
    a. Download the repository as a ZIP file.
    b. Unzip the downloaded folder on your computer.
2. **Create a new repository on your personal GitHub**

a. Create a brand new repository under your personal GitHub account.
b. Upload/push the contents of the unzipped starter repository into the new repository.

3. **Enable GitHub Actions to create pull requests**
   a. Open your repository and go to:
   Settings → Actions → General
   b. Scroll down to **Workflow permissions**.
   c. Select **Read and write permissions**.
   d. Enable **Allow GitHub Actions to create and approve pull requests**.
   e. Save your changes.
4. **Configure the OpenAI API key**
   a. Go to:
   Settings → Secrets and variables → Actions
   b. Create a new **Repository secret** named:

OPENAI_API_KEY

   c. Set its value to your OpenAI API key.
5. **Create the task-priority issue**
   a. Go to the **Issues** tab.
   b. Create a new issue for adding task priority using the requirements provided in the assignment.
6. **Create the workflow trigger label**
   a. Create a new GitHub label named:

approved

   b. Apply the approved label to the task-priority issue.
   c. Adding this label will trigger the GitHub Actions agent workflow.
7. **Complete the assignment**
   a. Follow the remaining instructions provided on Canvas:
   https://gatech.instructure.com/courses/558582/assignments/2656814